Cloud Resource Monitoring & Auto-Scaling System

Name – Ritu Jadhav

Objective:

To design and implement a Cloud Resource Monitoring and Auto-Scaling System using AWS that automatically monitors EC2 CPU utilization and dynamically scales resources up or down to ensure optimal performance, high availability, and cost efficiency.

Services Used:

1. **Amazon EC2 (Elastic Compute Cloud)** – To host and run virtual server instances
2. **Amazon Auto Scaling** – To automatically scale EC2 instances based on CPU utilization
3. **Amazon CloudWatch** – To monitor system performance and create CPU utilization alarms
4. **AWS Launch Template** – To define instance configuration for Auto Scaling
5. **Amazon Machine Image (AMI)** – To create a reusable server image
6. **Amazon SNS (Simple Notification Service)** – To send email alerts for monitoring events
7. **AWS IAM** – To manage secure access and permissions

Steps:

1.Create EC2 Instance –
To launch a basic EC2 instance that will act as the base server for auto-scaling.

2.Install Web Server & Stress Tool –
To configure the EC2 instance with Apache and CPU stress utility.

3.Enable Detailed Monitoring –
To enable enhanced CPU metrics for better scaling accuracy.

4.Create AMI (Amazon Machine Image)-
To create a reusable server image for Auto Scaling.

5.Create Launch Template –
To define how new EC2 instances should be launched automatically.

6.Create Auto Scaling Group –
To automatically manage EC2 instances based on load.

7.Create Scaling Policy (Target Tracking) –
To scale EC2 instances based on CPU utilization.

8.Create CloudWatch Alarms –
To monitor CPU usage and trigger scaling actions.

9.Test Auto Scaling –
To verify that scaling works correctly.

Implementation:

STEP 1: Launching an EC2 Instance

Theory:
Amazon EC2 (Elastic Compute Cloud) is a web service that provides scalable computing capacity in the AWS cloud. In this project, an EC2 instance is launched to act as the base server on which monitoring and auto-scaling mechanisms are implemented.

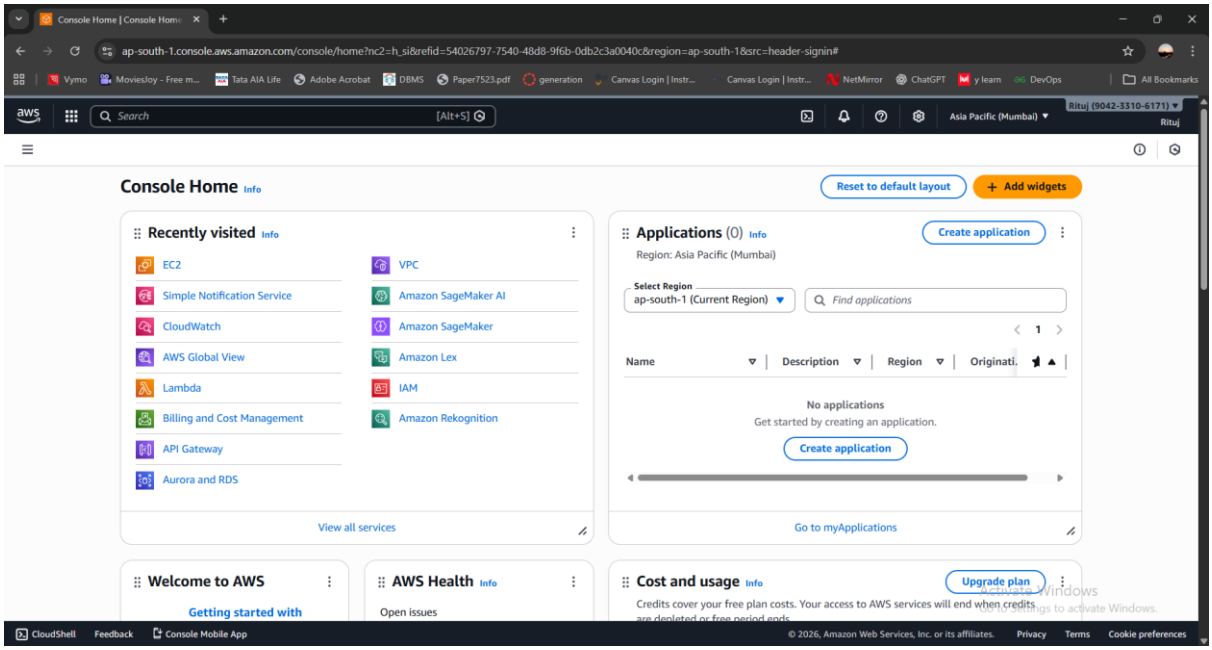
A basic EC2 instance running Amazon Linux is created using the AWS Management Console. This instance hosts a web server and generates CPU load for testing auto-scaling behavior.

Purpose:
To create a compute resource that can be monitored and scaled automatically based on CPU utilization.

1) **Login to AWS Management Console -**

• Open a web browser and navigate to the AWS Management Console.

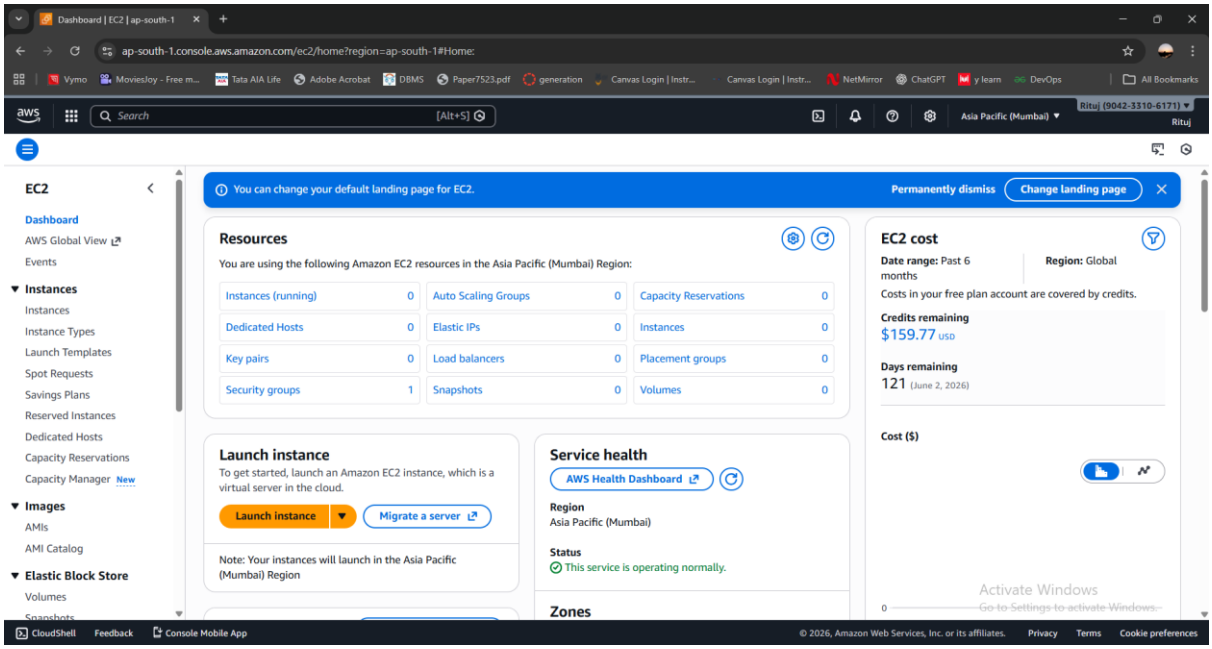
• Log in using valid AWS credentials.



2) **Navigate to EC2 Service-**

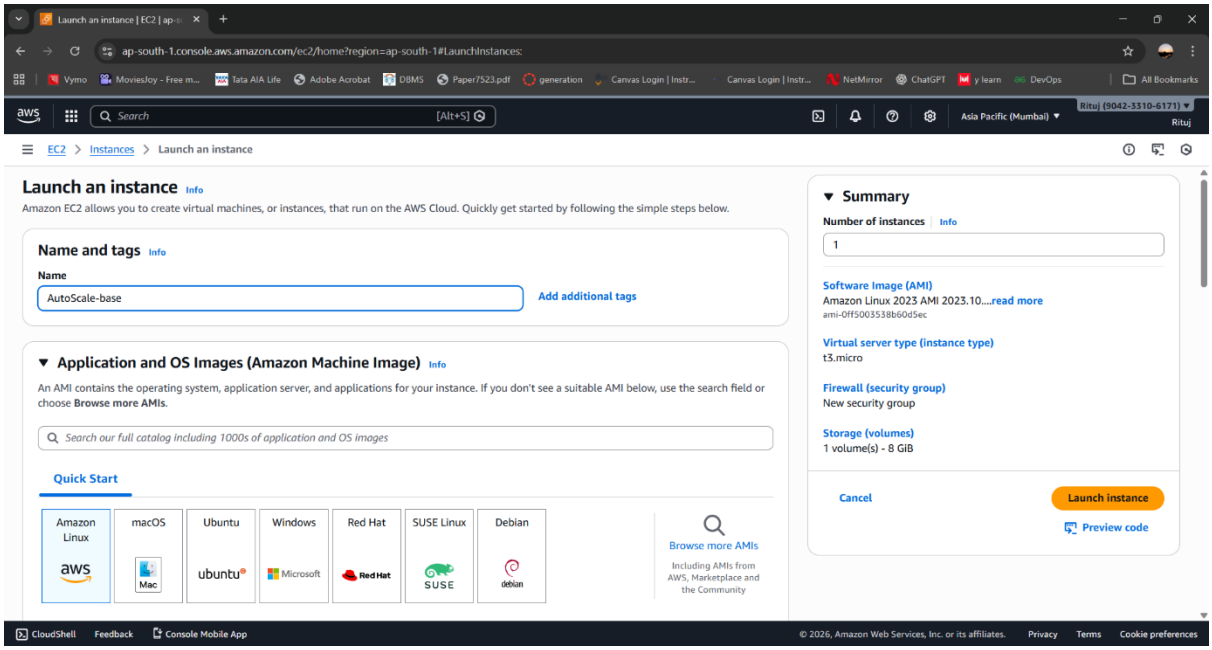
• From the AWS Console home page, search for EC2 in the search bar.

• Click on EC2 to open the EC2 Dashboard.



3) Launch a New EC2 Instance-

- In the EC2 Dashboard, click on Launch Instance & name it as **AutoScale-base**
- This opens the instance configuration wizard.

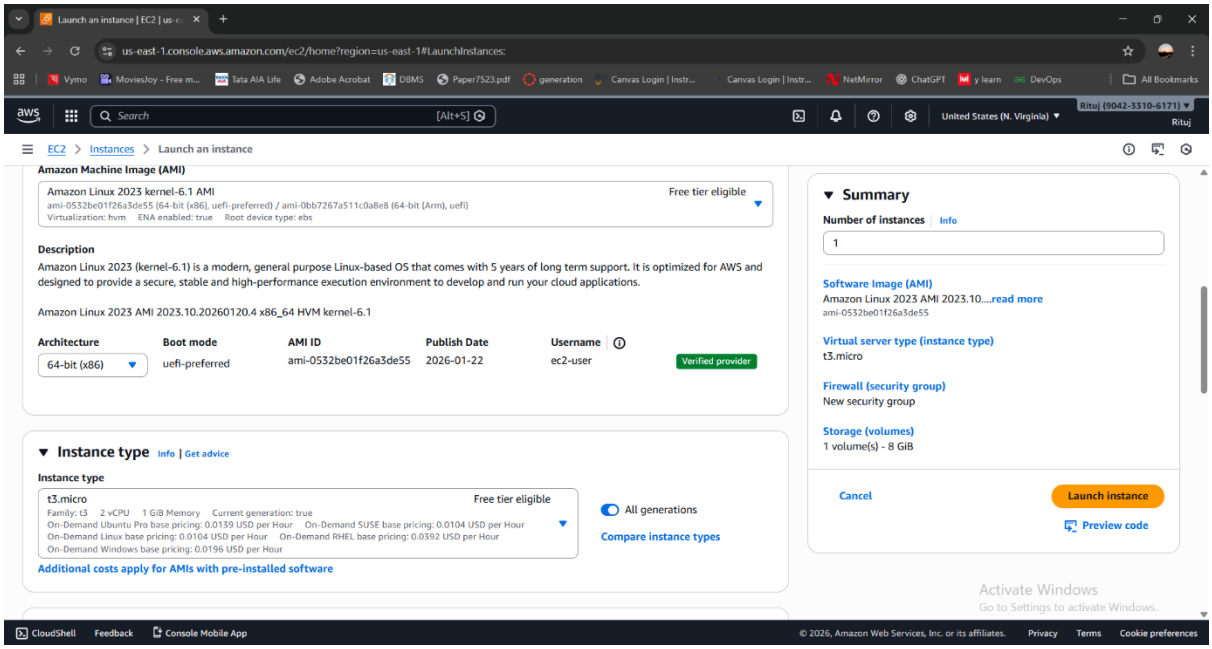


4) Select Amazon Machine Image (AMI)-

- Choose a Linux-based AMI such as Amazon Linux 2.
- This operating system is widely used, secure, and optimized for AWS services.

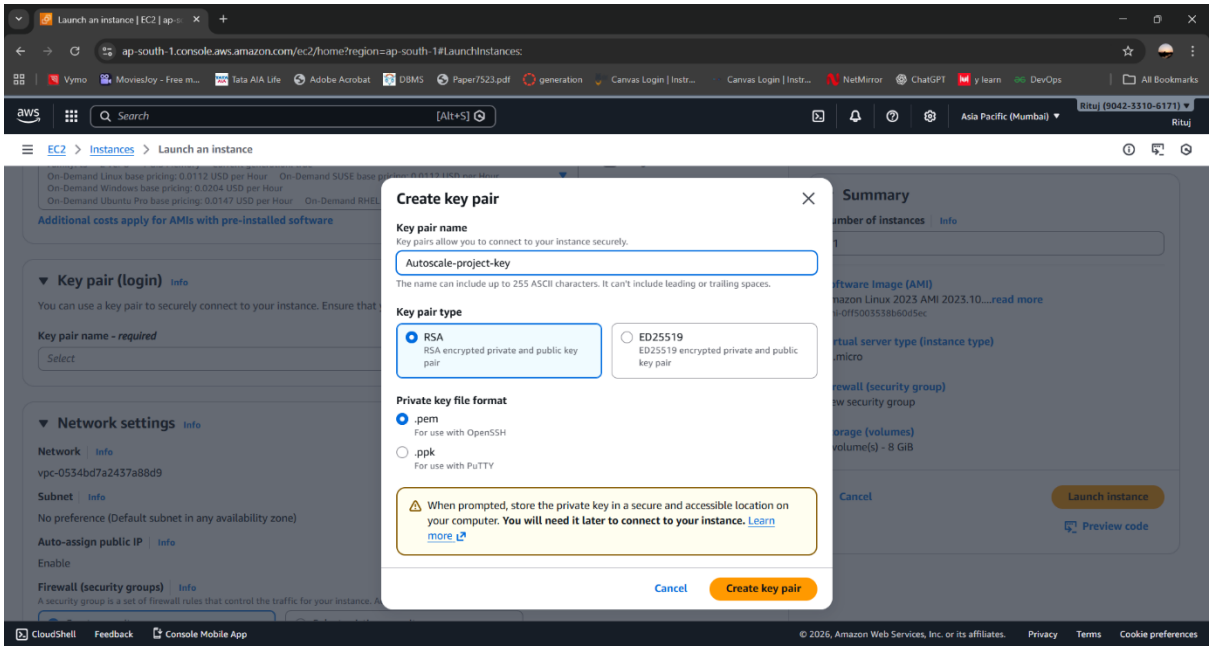
5) Choose Instance Type-

- Select an instance type based on application requirements (for example, t2.micro or t3.micro).
- This instance type provides sufficient CPU and memory for testing and practical purposes.



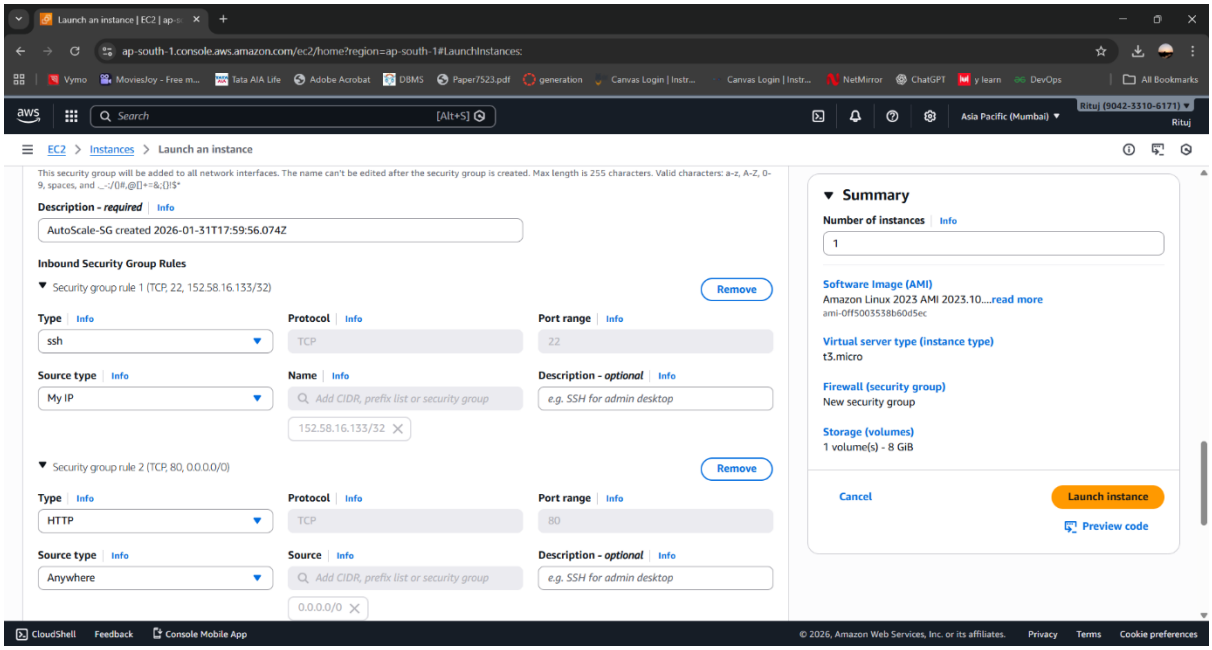
6) Configure Instance Details-

- Configure networking, subnet, and auto-assign public IP as required.
- Create a key pair **Autoscale-key**, which gets downloaded as **.pem** file
- Keep default settings if performing basic practical implementation.



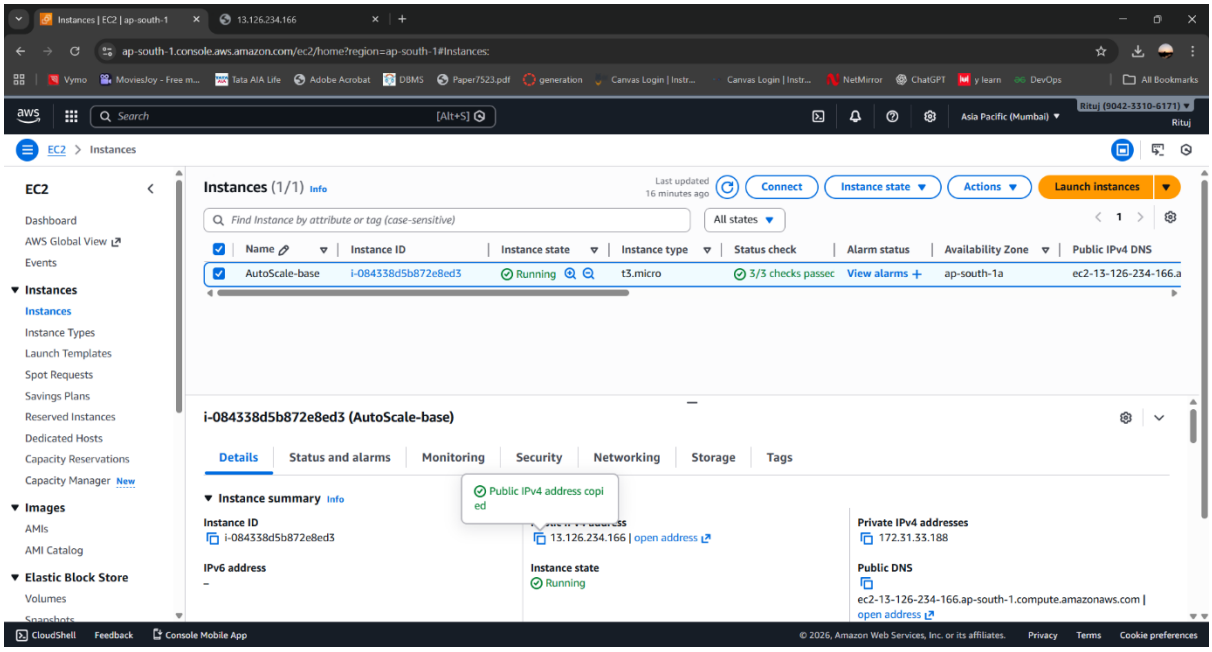
7) Configure Security Group-

- Create a security group as **AutoScale-SG.**
- Allow required inbound rules such as: **SSH -22/MyIP** & **HTTP – 80/Anywhere**



8) Review and Launch-

- Review all instance configuration details.
- Click on Launch.



STEP 2: Configuring the EC2 Instance

Theory:
After launching the EC2 instance, it is configured by installing a web server (Apache) and a stress tool. Apache helps verify that the instance is accessible, while the stress tool is used to artificially increase CPU usage for testing scaling policies.

The instance is accessed securely using SSH, and required packages are installed using Linux package managers.

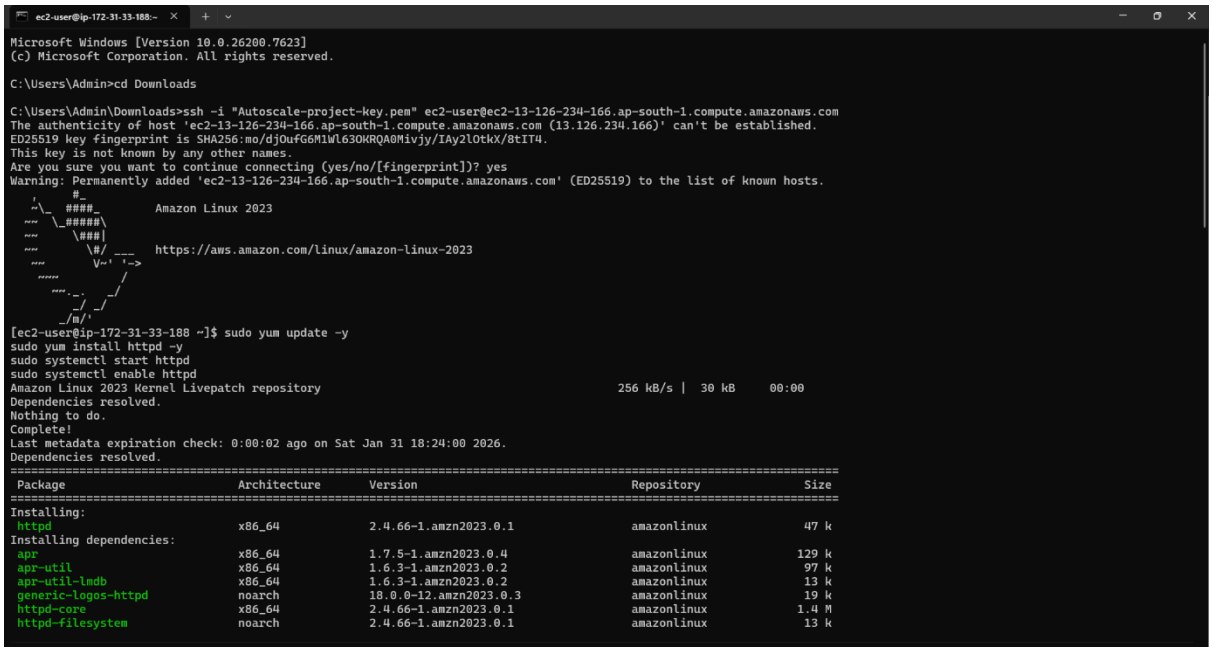
Purpose:
To prepare the EC2 instance for monitoring and CPU load testing.

1) **Login to EC2 Instance-**

   - **ssh -i key.pem ec2-user@<public IP>**

2) **Install a Simple Web App (Load Generator)-**
   - We create CPU load easily using Apache
   - Write below commands:

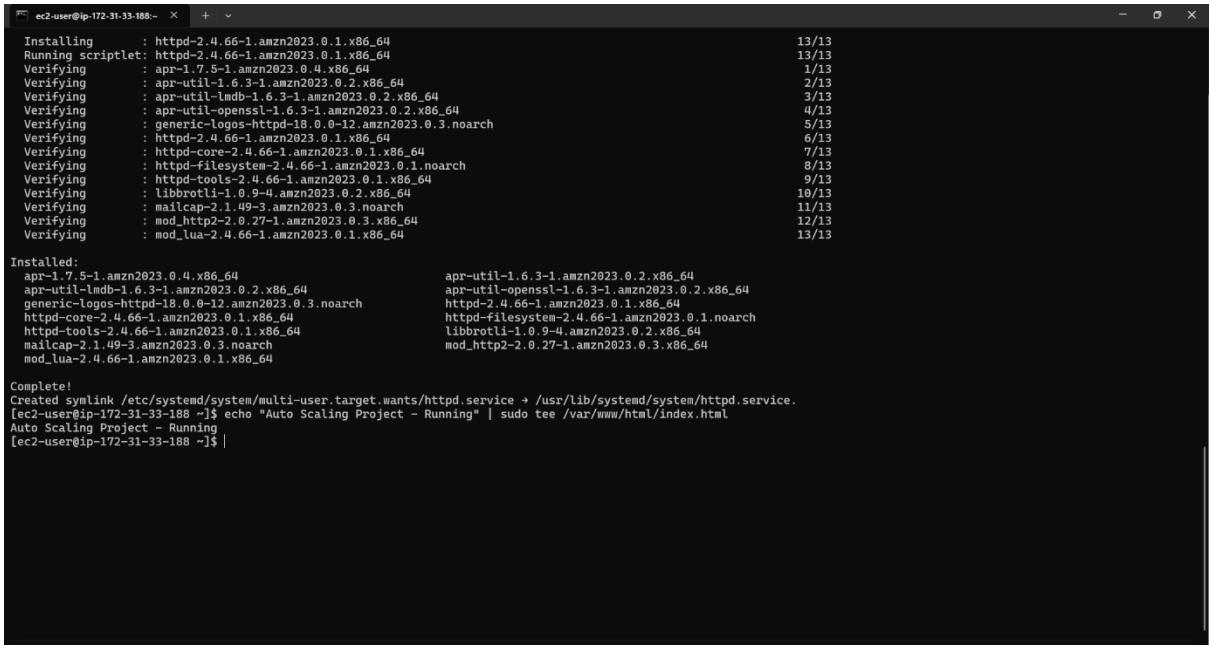
     sudo yum update -y
     sudo yum install httpd -y
     sudo systemctl start httpd
     sudo systemctl enable httpd

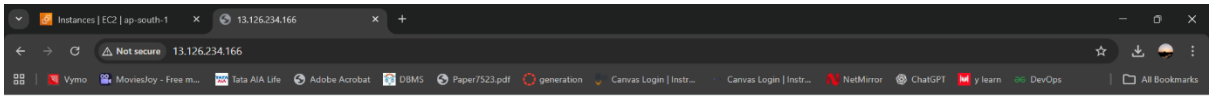


3) **Create simple webpage-**

   - Enter this below command to check the connection is built successfully or not.

echo "Auto Scaling Project - Running" | sudo tee /var/www/html/index.html





STEP 3: Enabling Monitoring using CloudWatch

Theory:
Amazon CloudWatch is a monitoring service that collects and tracks metrics, logs, and events. By default, EC2 instances provide basic monitoring. For better accuracy, detailed monitoring is enabled, which collects CPU utilization metrics at 1-minute intervals.

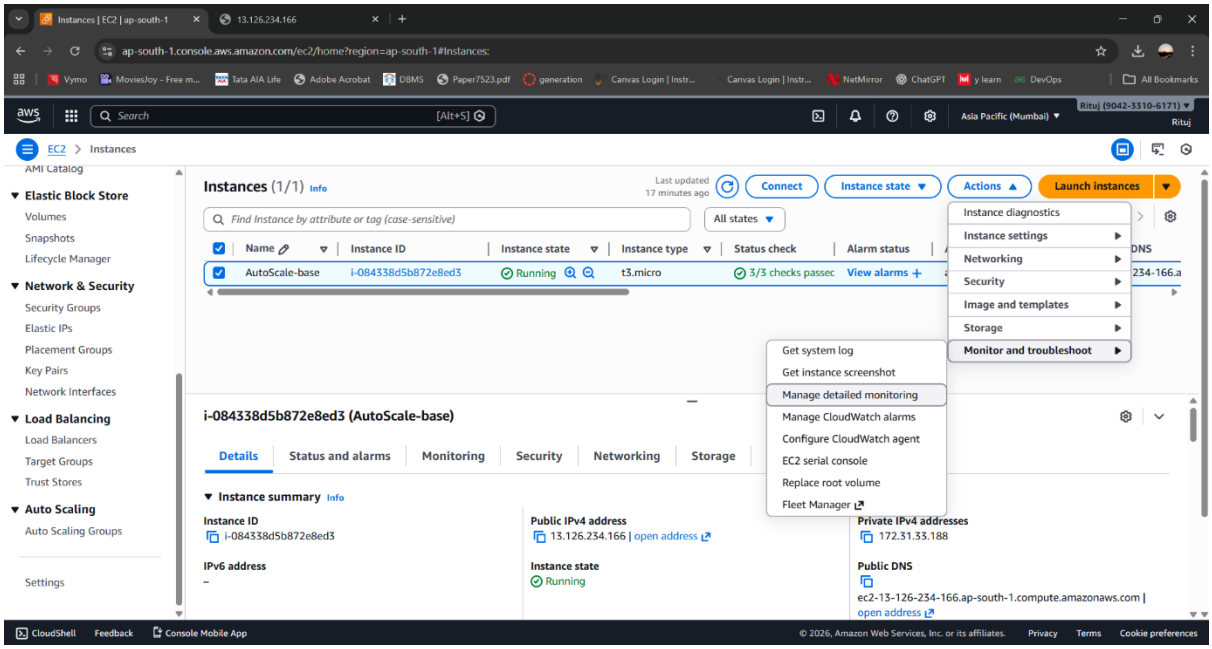
These metrics are crucial for triggering auto-scaling actions based on real-time CPU usage.

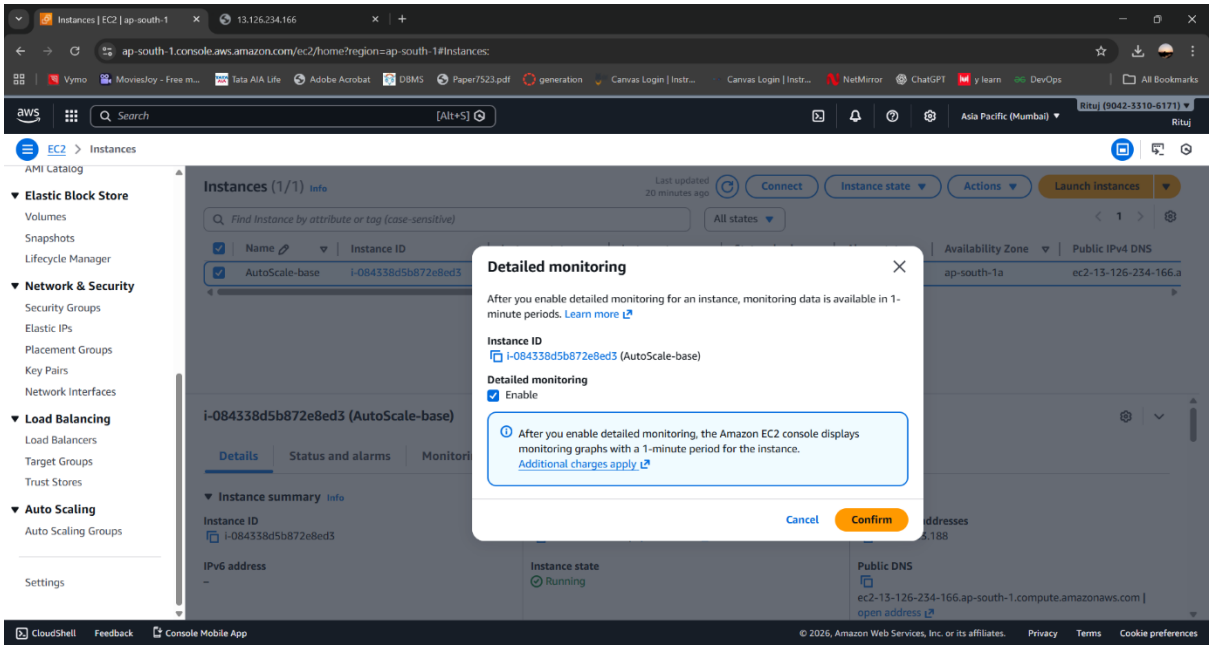
Purpose:
To monitor EC2 CPU utilization continuously.

1) Enable Detailed Monitoring-

- EC2 → Instance → Actions

- Monitoring → Enable detailed monitoring
- Makes CloudWatch more accurate



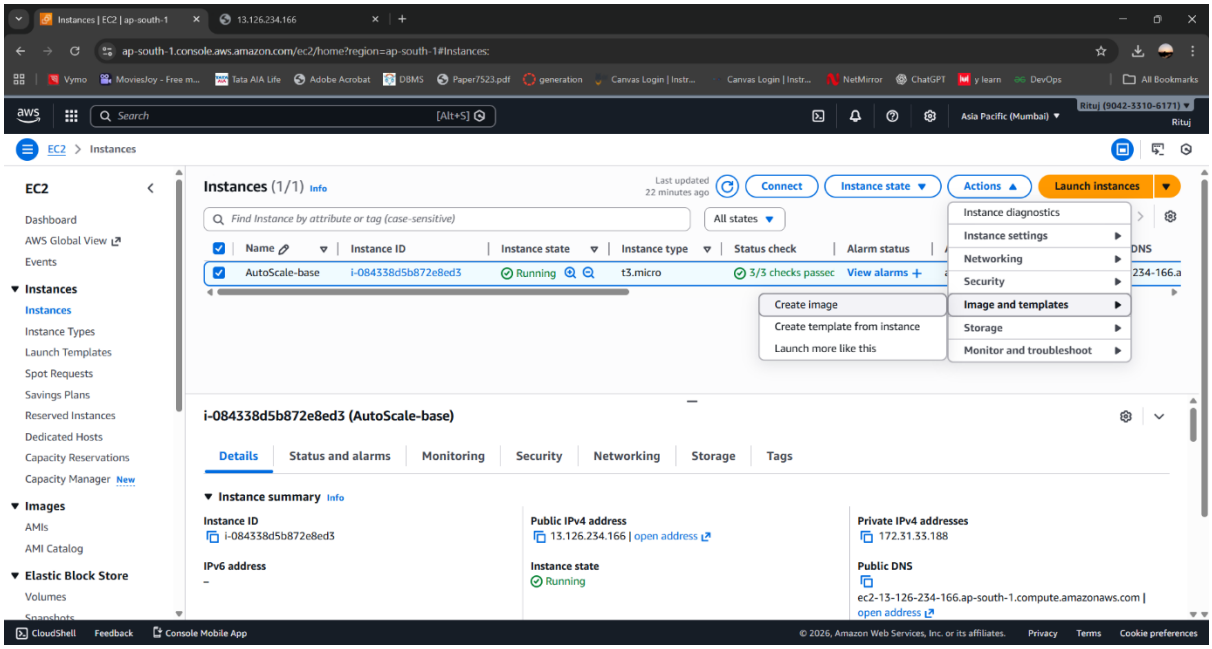


STEP 4: Creating an Amazon Machine Image (AMI)

Theory:
An Amazon Machine Image (AMI) is a snapshot of a configured EC2 instance. It includes the operating system, installed software, and configurations.

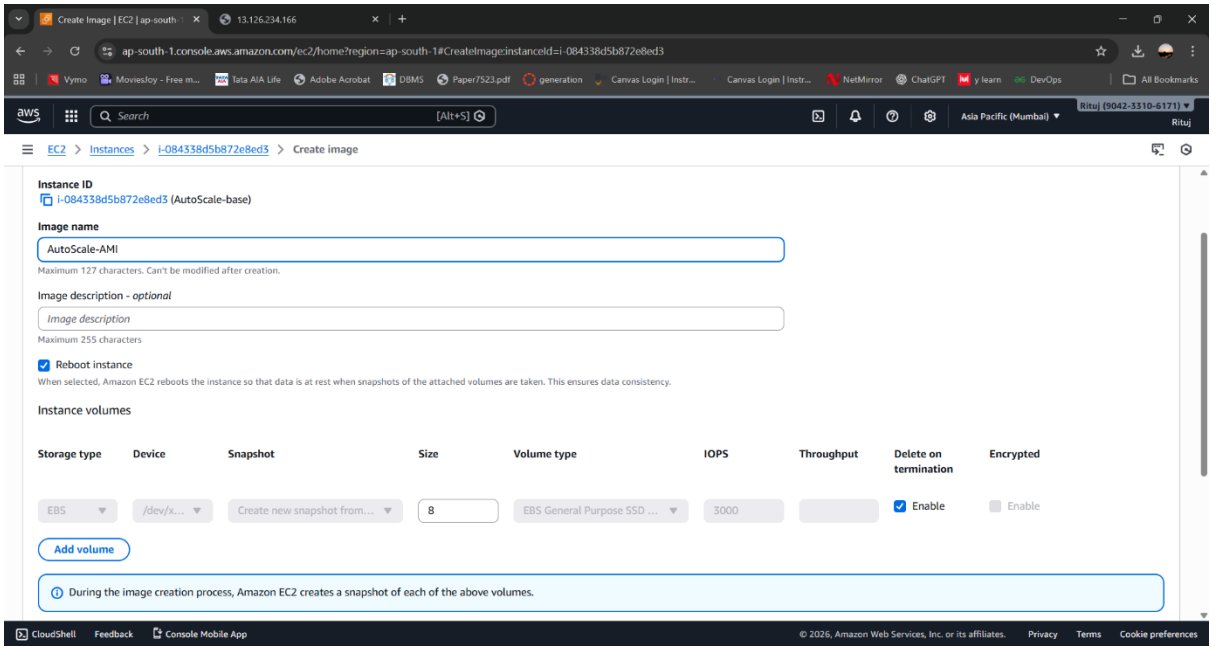
In this project, an AMI is created from the configured EC2 instance. This AMI is later used by the Auto Scaling Group to launch identical instances automatically.

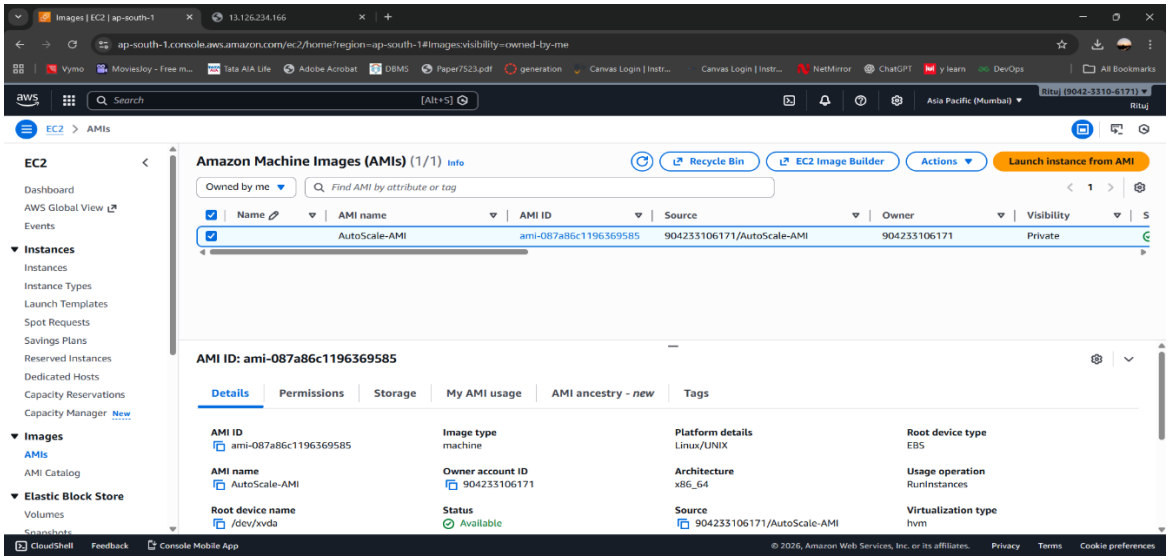
Purpose:
To create a reusable image for scaling EC2 instances.



1) Create AMI-

- Name it as **AutoScale-AMI**



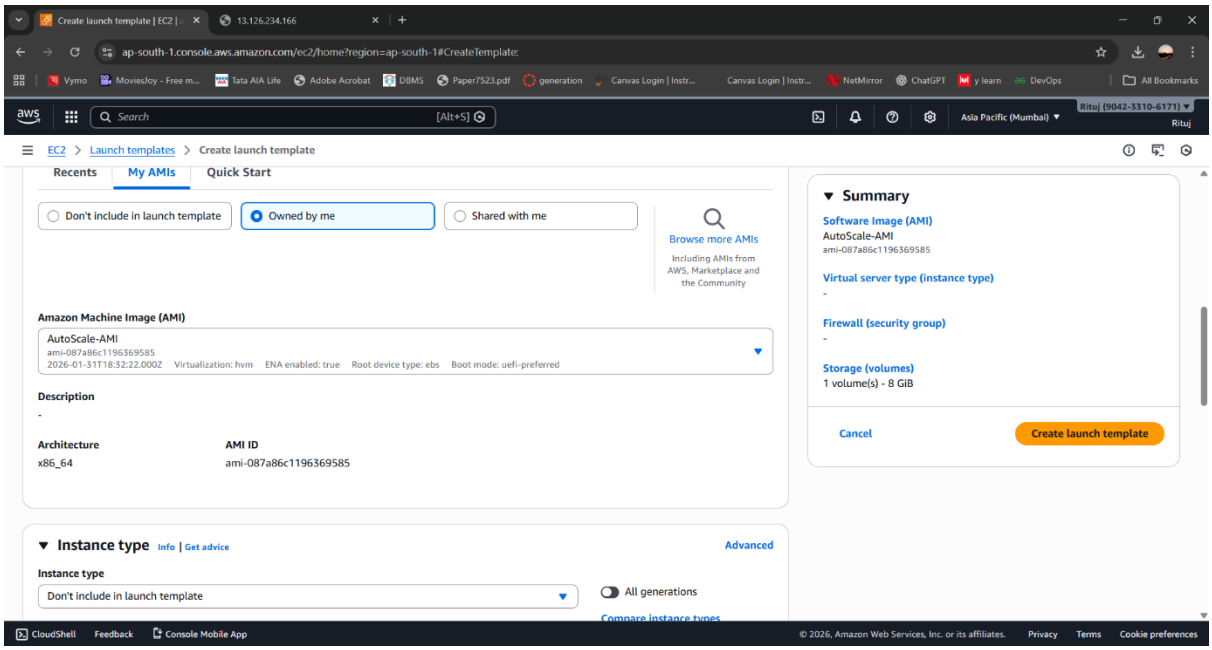


STEP 5: Creating a Launch Template

Theory:
A Launch Template defines the configuration for EC2 instances launched by the Auto Scaling Group. It includes details such as AMI ID, instance type, security group, and key pair.

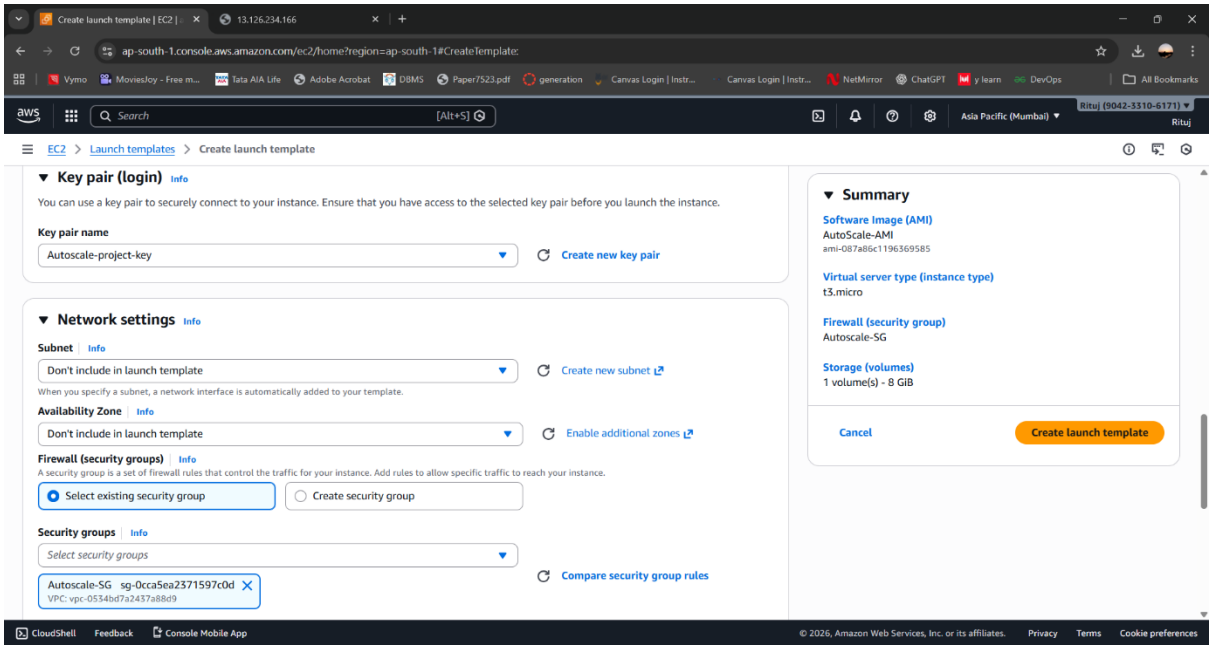
Using a launch template ensures consistency across all auto-scaled instances.

Purpose:
To standardize instance configuration for auto-scaling.



1) **After selecting Network Setting-**

   - Click on **create launch template**



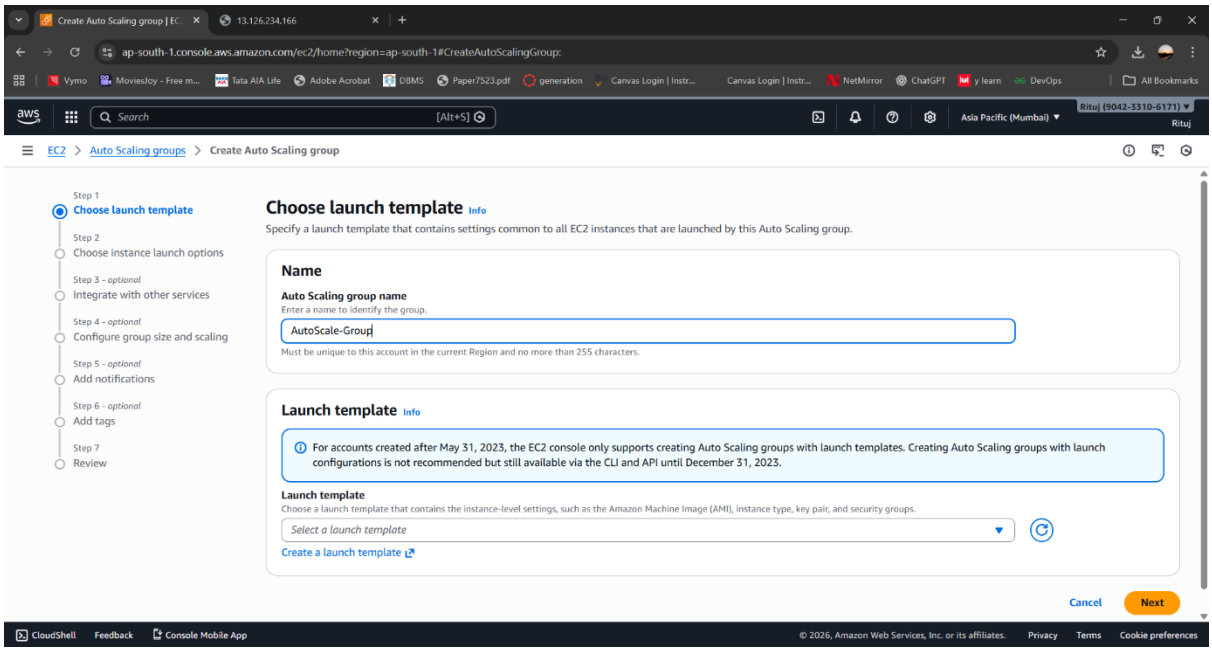
STEP 6: Creating an Auto Scaling Group

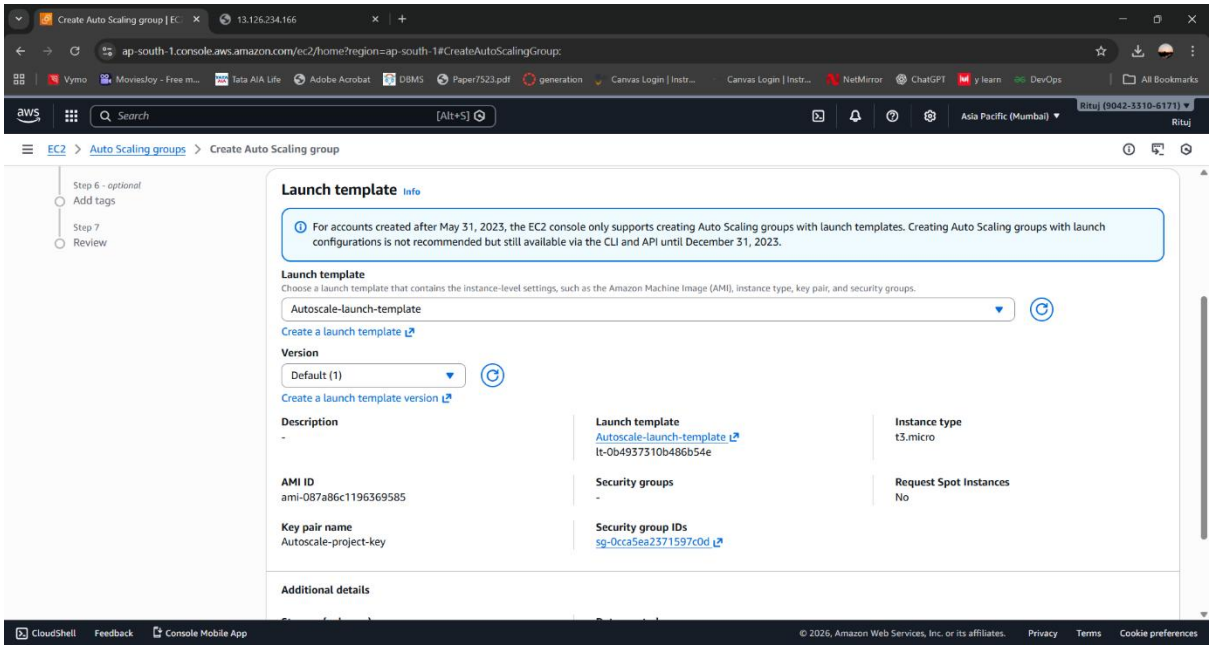
Theory:
An Auto Scaling Group (ASG) automatically manages the number of EC2 instances based on defined conditions. It ensures high availability and optimal performance by scaling resources up or down.

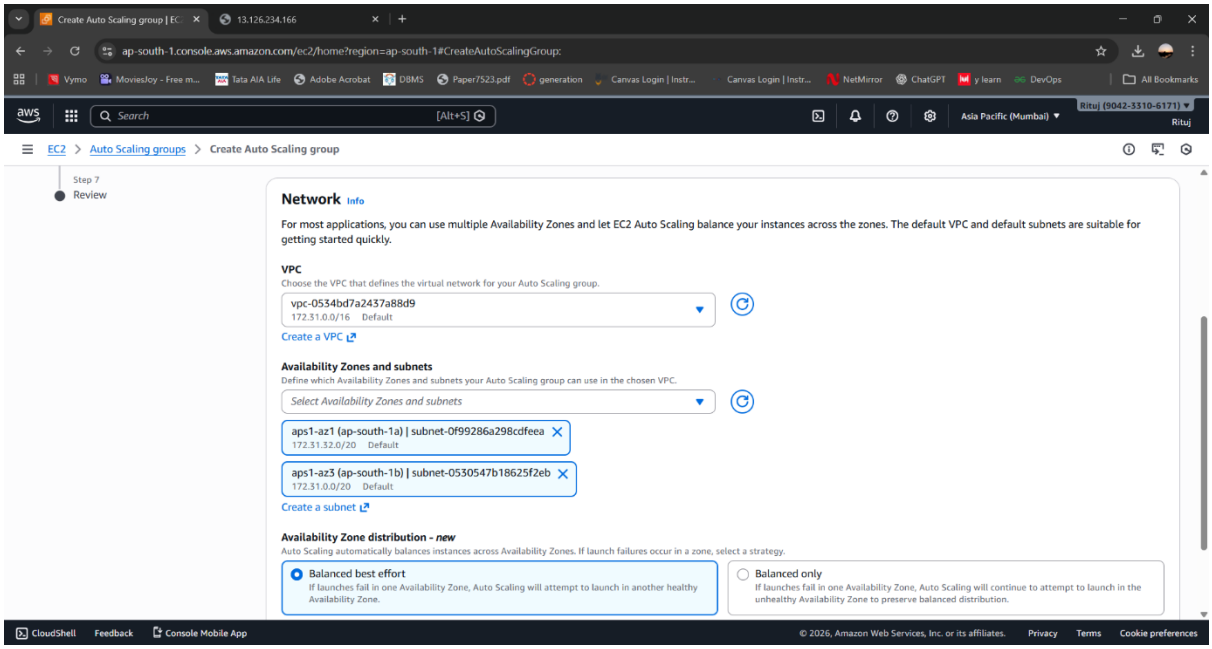
The Auto Scaling Group is configured with:

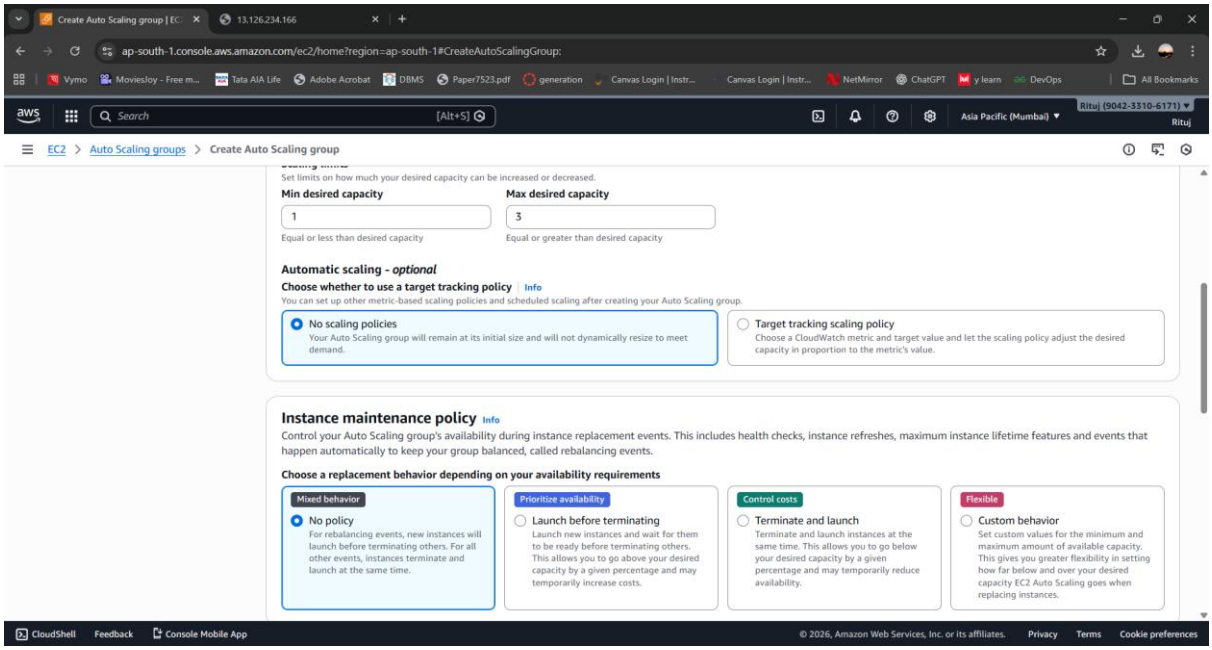
- Minimum instances: 1
- Desired instances: 1
- Maximum instances: 3
- Multiple Availability Zones for fault tolerance

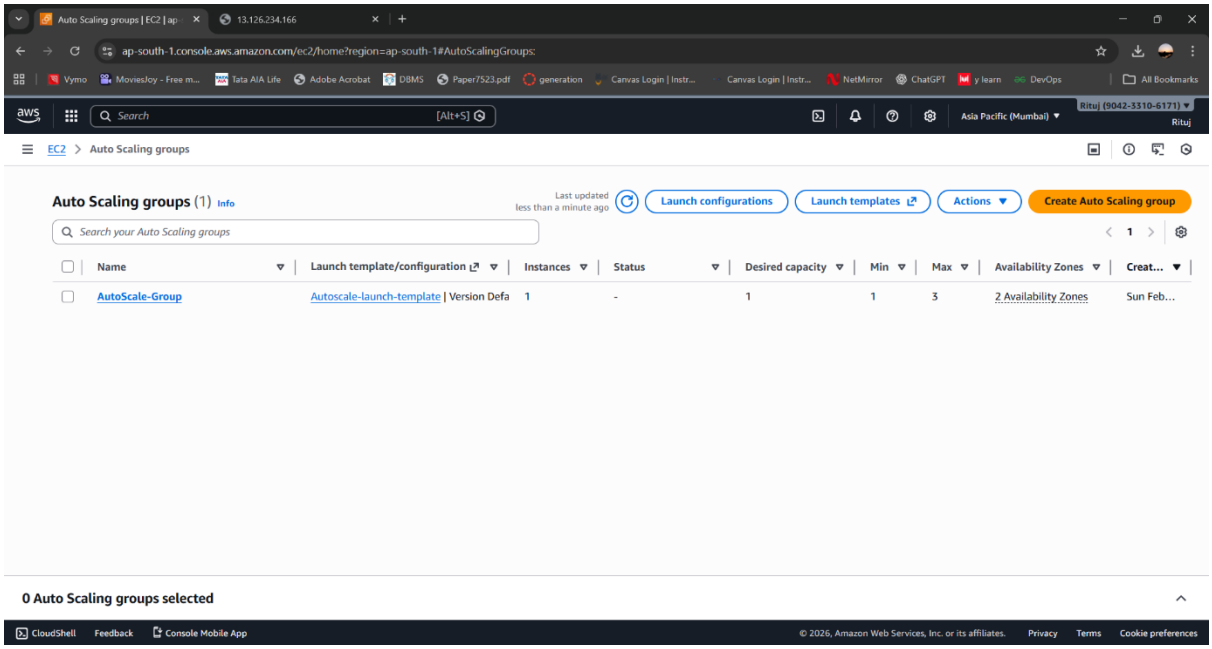
Purpose:
To dynamically manage EC2 instances based on workload.




Create Auto Scaling group | EC
13.126.234.166
ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#CreateAutoScalingGroup:
Search
[Alt+S]
Asia Pacific (Mumbai)
Rituj (9042-3310-6171)
Rituj
EC2 > Auto Scaling groups > Create Auto Scaling group
Step 6 - optional
Add tags
Step 7
Review
Launch template Info
For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.
Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.
Autoscale-launch-template
Create a launch template
Version
Default (1)
Create a launch template version
Description
-
Launch template
Autoscale-launch-template
lt-0b4937310b486b54e
Instance type
t3.micro
AMI ID
ami-087a86c1196369585
Security groups
-
Request Spot Instances
No
Key pair name
Autoscale-project-key
Security group IDs
sg-0cca5ea2371597c0d
Additional details
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Create Auto Scaling group | EC
13.126.234.166
ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#CreateAutoScalingGroup:
Search
[Alt+S]
Asia Pacific (Mumbai)
Rituj (9042-3310-6171)
Rituj
EC2 > Auto Scaling groups > Create Auto Scaling group
Step 7
Review
Network Info
For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.
VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.
vpc-0534bd7a2437a88d9
172.31.0.0/16 Default
Create a VPC
Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.
Select Availability Zones and subnets
aps1-az1 (ap-south-1a) | subnet-0f99286a298cdfeea
172.31.32.0/20 Default
aps1-az3 (ap-south-1b) | subnet-0530547b18625f2eb
172.31.0.0/20 Default
Create a subnet
Availability Zone distribution - new
Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.
Balanced best effort
If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.
Balanced only
If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Create Auto Scaling group | EC
13.126.234.166
ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#CreateAutoScalingGroup:
Search
[Alt+S]
Asia Pacific (Mumbai)
Rituj (9042-3310-6171)
Rituj
EC2 > Auto Scaling groups > Create Auto Scaling group
Set limits on how much your desired capacity can be increased or decreased.
Min desired capacity
1
Equal or less than desired capacity
Max desired capacity
3
Equal or greater than desired capacity
Automatic scaling - optional
Choose whether to use a target tracking policy Info
You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.
No scaling policies
Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.
Target tracking scaling policy
Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.
Instance maintenance policy Info
Control your Auto Scaling group's availability during instance replacement events. This includes health checks, instance refreshes, maximum instance lifetime features and events that happen automatically to keep your group balanced, called rebalancing events.
Choose a replacement behavior depending on your availability requirements
Mixed behavior
No policy
For rebalancing events, new instances will launch before terminating others. For all other events, instances terminate and launch at the same time.
Prioritize availability
Launch before terminating
Launch new instances and wait for them to be ready before terminating others. This allows you to go above your desired capacity by a given percentage and may temporarily increase costs.
Control costs
Terminate and launch
Terminate and launch instances at the same time. This allows you to go below your desired capacity by a given percentage and may temporarily reduce availability.
Flexible
Custom behavior
Set custom values for the minimum and maximum amount of available capacity. This gives you greater flexibility in setting how far below and over your desired capacity EC2 Auto Scaling goes when replacing instances.
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

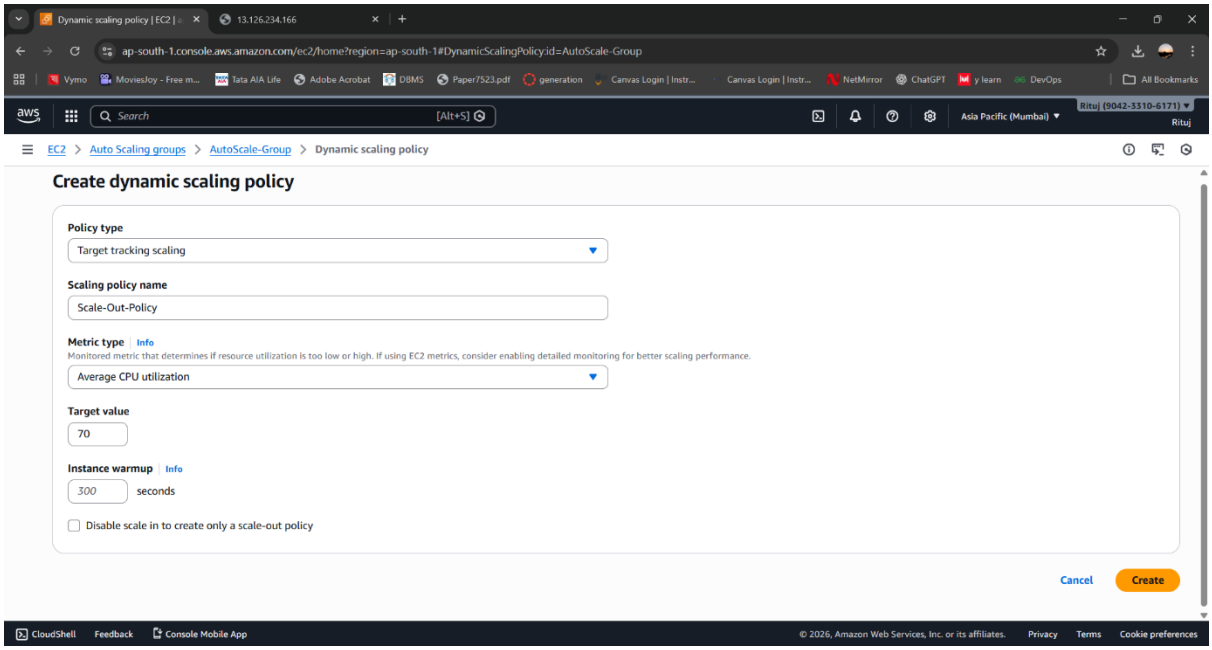


STEP 7: Configuring Scaling Policies

Theory:
Scaling policies define when and how EC2 instances are scaled. In this project, **Target Tracking Scaling** is used, where AWS automatically adjusts capacity to maintain a target CPU utilization.

A target CPU utilization of **70%** is set. When CPU usage exceeds this value, new instances are launched. When CPU usage drops below the target, instances are terminated automatically.

Purpose:
To automate scale-out and scale-in actions.



STEP 8: CloudWatch Alarms (Auto-Generated)

Theory:
When a target tracking policy is applied, AWS automatically creates CloudWatch alarms. These alarms monitor CPU utilization and trigger scaling actions.

Two alarms are created internally:

- **Scale-out alarm** – when CPU exceeds the target
- **Scale-in alarm** – when CPU falls below the target

These alarms are managed by AWS and cannot be manually edited.

Purpose:
To trigger auto-scaling actions based on CPU metrics.

1) **Monitor CPU usage-**

   • Go to **CloudWatch**

   • Create **Alarm**

   • Select Metric: EC2 → Per Instance Metrics→ CPUUtilization

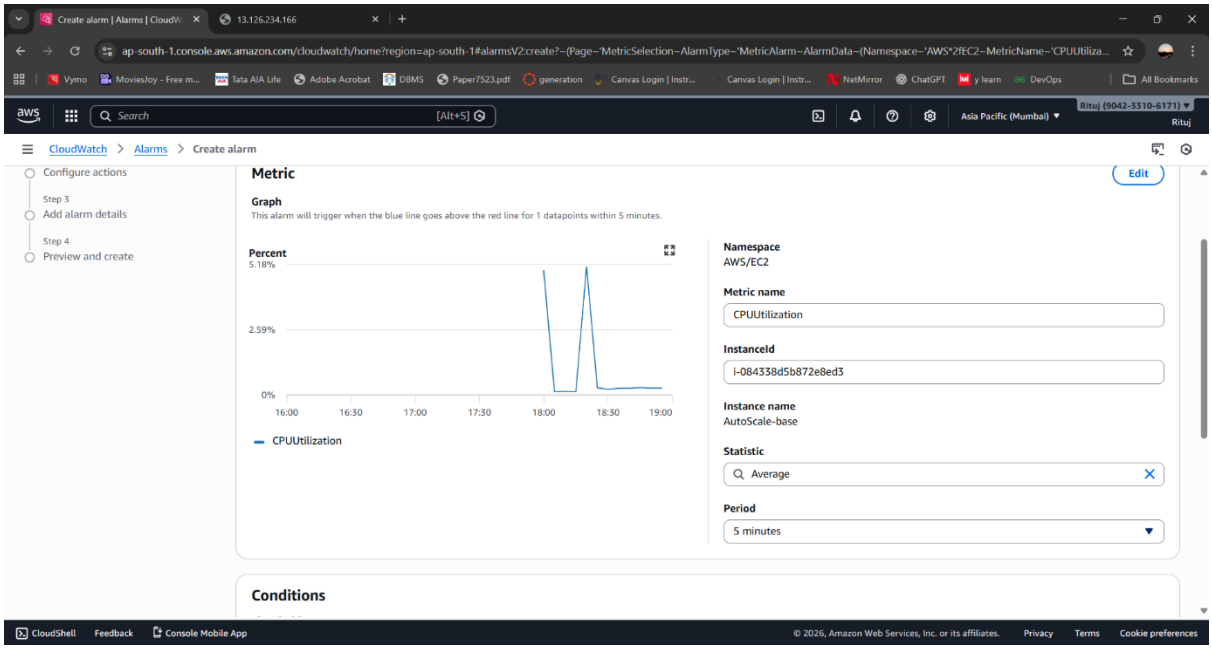
   • Condition: CPU > **70% for 2 minutes**

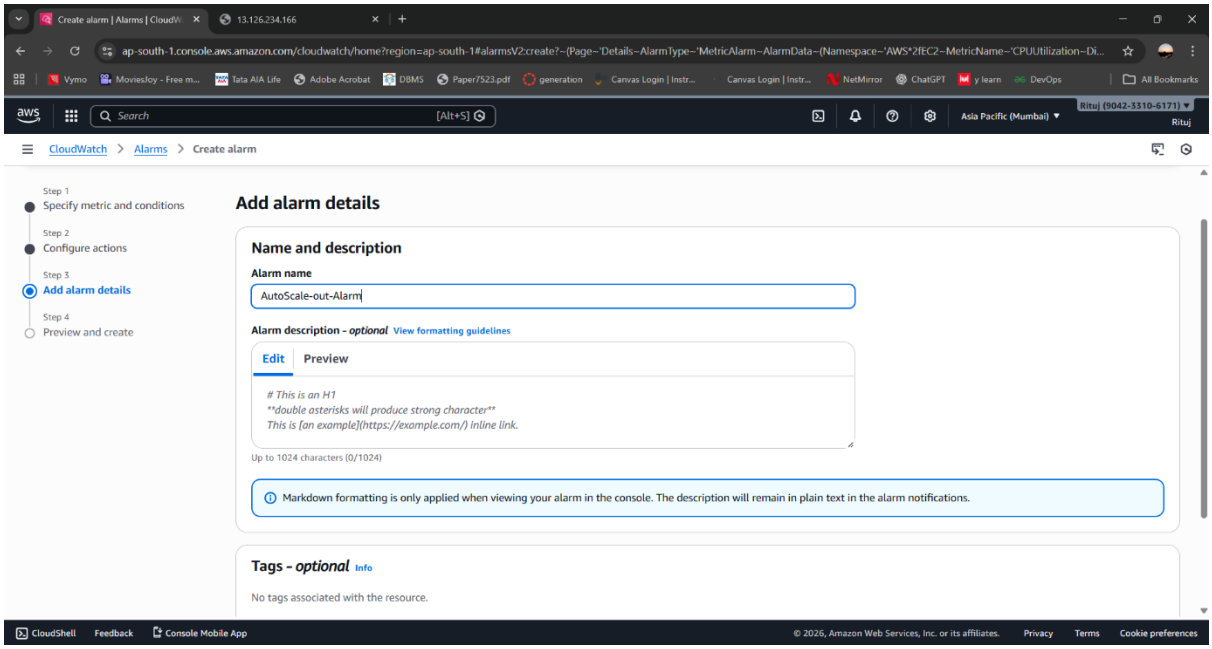
   • Action: Trigger **Auto Scaling Policy (Scale Out)**

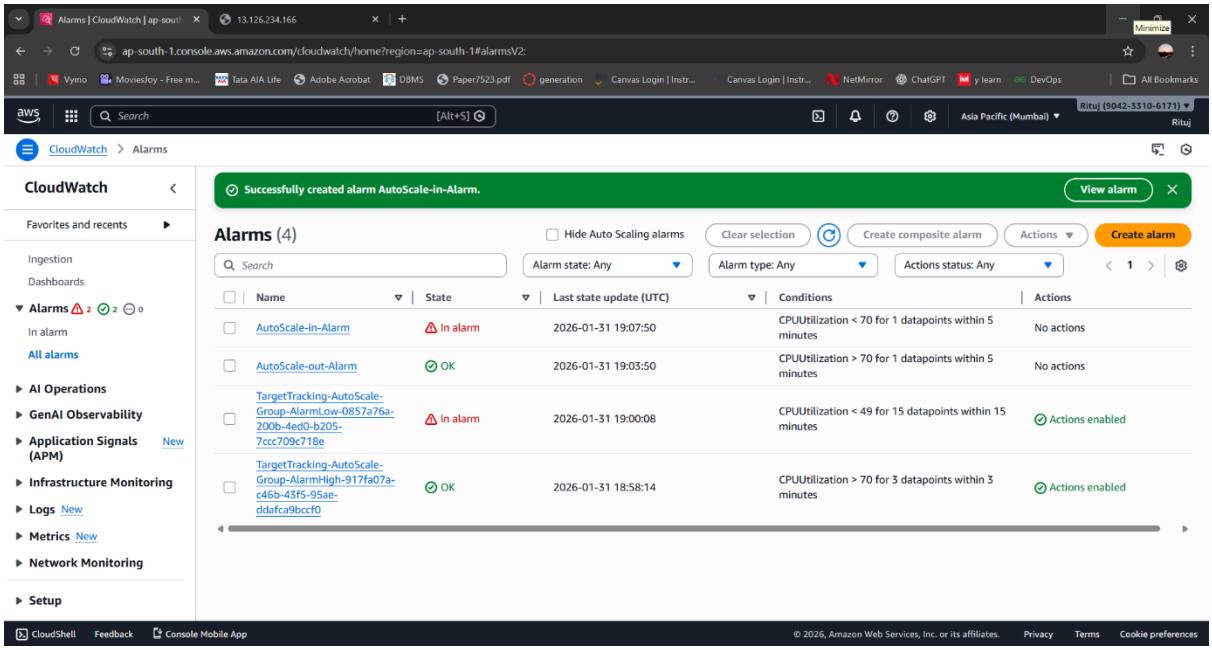
2) **Create Another Alarm-**

   • CPU < **30%**

   • Trigger **Scale In**





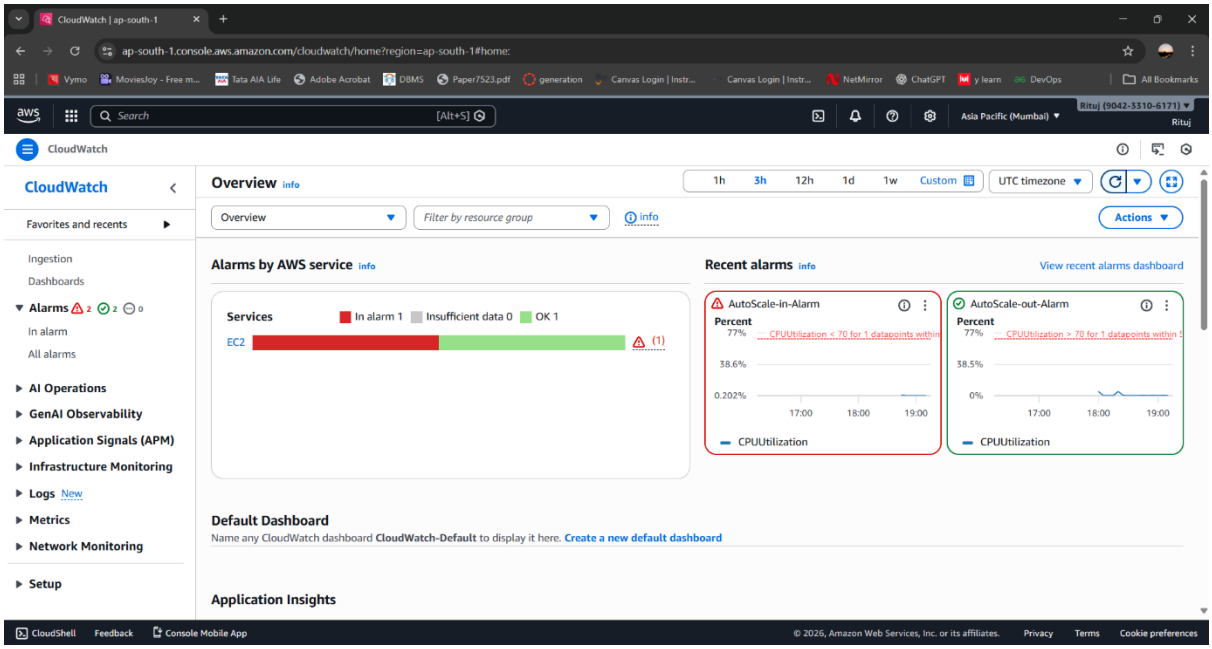


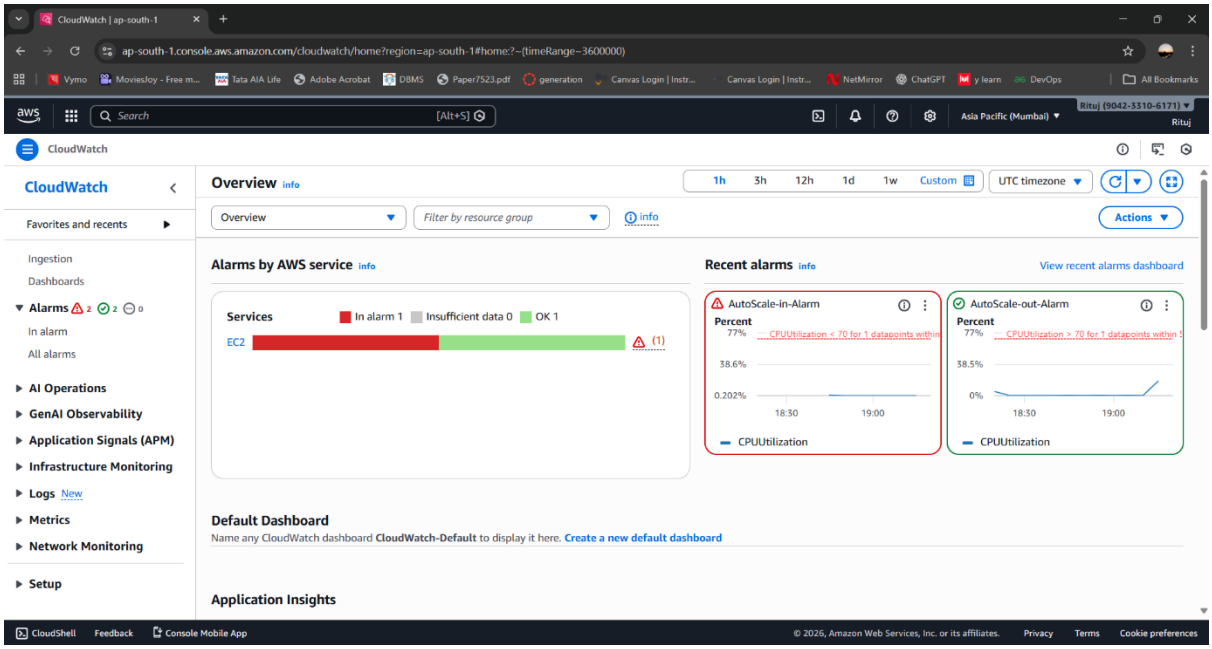
STEP 9: Generating CPU Load

Theory:
To test the auto-scaling system, artificial CPU load is generated using the stress tool. This simulates real-world traffic conditions and validates whether the scaling policy responds correctly.

When CPU utilization increases beyond 70%, the Auto Scaling Group launches additional EC2 instances automatically.

Purpose:
To verify that auto-scaling works correctly.



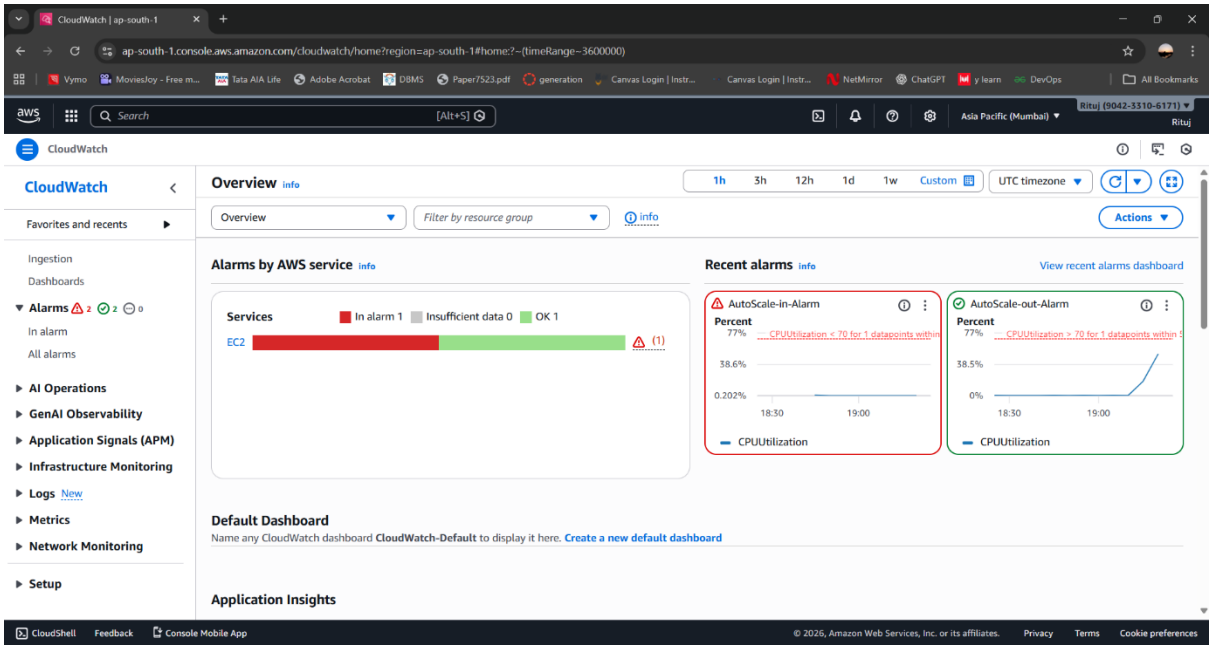


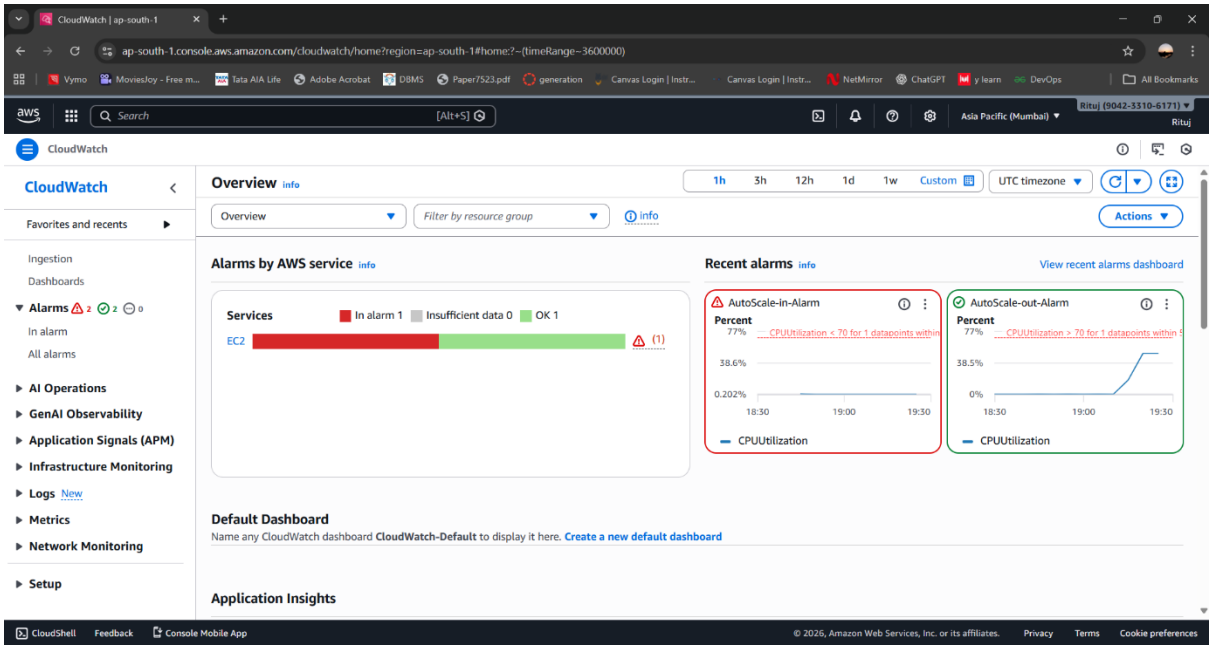
STEP 10: Observing CPU Utilization Output

Theory:
CloudWatch dashboards display real-time CPU utilization metrics. When CPU load increases, the metric graph shows a spike, triggering the scale-out alarm.

As CPU usage decreases, the scale-in alarm activates, and instances are reduced while maintaining the minimum capacity.

Purpose:
To observe monitoring and scaling behavior in real time.





Final Output:

Result:

- EC2 instances are automatically scaled based on CPU utilization
- CloudWatch continuously monitors performance
- Auto Scaling ensures high availability and cost efficiency
- The system adapts dynamically to workload changes

This project demonstrates **real-time cloud monitoring, automation, and scalability using AWS services**.